STUDENT ASSIGNMENT TRACKER

10-Minute Project Explanation & Demo Script

Presenter: Bhalchandra Rana
Project: Student Assignment Tracker
Duration: Approximately 10 minutes

---

1. INTRODUCTION — 0:00–1:00

[SCREEN]

Show the project landing page.

[SPEAK]

Hello everyone.

My name is **Bhalchandra Rana**, and today I am going to demonstrate my full-stack project called **Student Assignment Tracker**.

The main purpose of this project is to provide students with a centralized platform to manage their assignments, deadlines, submissions, and academic progress.

As students, we often receive assignments from multiple subjects and have to keep track of different deadlines.

This can become difficult when information is spread across notebooks, messages, or different applications.

So, I developed this application with the idea of bringing these activities together into a single platform.

The project consists of a **React.js frontend**, a **Java Spring Boot backend**, and a **MySQL database**.

---

2. PROBLEM & SOLUTION — 1:00–2:00

[SCREEN]

Scroll through the landing page.

[SPEAK]

The problem I wanted to address was simple:

How can a student easily keep track of assignments and their academic activities from one place?

The application provides a dedicated interface where students can manage and monitor their academic tasks.

The landing page provides the entry point to the application.

From here, the user can navigate to the login page and access the student dashboard.

The UI is designed to be simple and responsive so that the important information is easy to access.

The main areas of the application include assignments, submissions, courses, calendar information, and the student dashboard.

---

3. TECHNOLOGY STACK — 2:00–3:00

[SCREEN]

Open README / project architecture or display the technology stack.

[SPEAK]

For the frontend, I used:

- **React.js**
- **Vite**
- **React Router**
- **Lucide React**

For the backend, I used:

- **Java 21**
- **Spring Boot 3.5**
- **Spring Data JPA**
- **Hibernate**
- **Maven**

For security, I implemented:

- **Spring Security**
- **JWT authentication**
- **BCrypt password hashing**
- **Role-based authorization**

For persistence, I used:

- **MySQL**
- **Flyway**
- **Docker**

I selected this technology stack because it allowed me to implement a complete full-stack application while also following a structure similar to modern Java backend applications.

---

4. SYSTEM ARCHITECTURE — 3:00–5:00

[SCREEN]

Show the architecture diagram from README.

[SPEAK]

Now I would like to explain the architecture of the application.

The frontend communicates with the backend using **REST APIs**, with JSON as the data format.

The overall flow is:

React → Controller → Service → Repository → MySQL

The React frontend sends an HTTP request to the Spring Boot backend.

The request first reaches the **Controller layer**.

Controllers are kept thin. Their main responsibility is handling HTTP requests, validating input, and returning responses.

The Controller communicates with the **Service layer**.

The Service layer contains the application's business logic.

The Service then communicates with the **Repository layer**, which uses Spring Data JPA and Hibernate to interact with MySQL.

I have also separated **DTOs, entities, mappers, security, configuration, and exception handling** into their own packages.

One important design decision is that I don't expose JPA entities directly through the APIs.

Instead, the controllers work with DTOs.

This keeps the API contract separate from the database model.

For dependency injection, I use **constructor injection**.

I have also disabled Open Session in View so that database access remains within the appropriate application layers.

For database schema management, I use **Flyway migrations**, while Hibernate is configured to validate the mappings rather than modify the database schema.

---

5. STARTING THE APPLICATION — 5:00–5:30

[SCREEN]

Open PowerShell.

[COMMAND]

.\start-all.ps1

[SPEAK]

I have also created startup scripts to simplify the development process.

Instead of manually starting each service, I can start the complete application using a single PowerShell command.

This starts or verifies the MySQL database and then starts the Spring Boot backend and React frontend.

The frontend runs on:

http://localhost:5173

and the backend runs on:

http://localhost:8081

The MySQL database is running inside Docker.

---

6. LANDING PAGE — 5:30–6:00

[SCREEN]

Browser → Landing page.

[SPEAK]

This is the landing page of the application.

Here we have the application name, navigation menu, and the main introduction to the system.

The main call-to-action takes the user toward the application.

I have kept the design clean so that the user can quickly understand what the application is intended for.

Now I will move to the login process.

---

7. LOGIN & JWT AUTHENTICATION — 6:00–7:00

[SCREEN]

Navigate to Login.

[ENTER]

Email: student@example.com

Password: Student@123

[SPEAK]

The application uses database-backed authentication.

For this demonstration, I am using the development student account provided by the application.

When I submit the login form, the credentials are sent to the Spring Boot authentication endpoint.

The backend verifies the credentials.

Passwords are stored using **BCrypt hashing**, rather than plain text.

After successful authentication, the backend generates a signed **JWT access token**.

The frontend then uses this token when making authenticated requests.

The authentication is stateless, meaning the backend does not need to maintain a traditional server-side session.

The project also supports role-based authorization.

For example, protected endpoints can require a specific role such as ROLE_ADMIN.

---

8. DASHBOARD — 7:00–8:00

[SCREEN]

Open Dashboard.

[SPEAK]

After successful login, the student is taken to the dashboard.

The dashboard is designed to provide a quick overview of the student's academic activities.

Here we have sections for:

- Total assignments

- Pending work
- Submitted assignments
- Overdue work
- Upcoming deadlines
- Recent submissions
- Calendar information
- Messages

The idea behind the dashboard is to make the most important information visible without requiring the student to navigate through multiple pages.

At the current stage, the landing page, login flow, dashboard UI, authentication, authorization, database infrastructure, and application architecture are implemented.

The assignment-domain APIs and live dashboard data are planned as the next development phase.

---

9. BACKEND CODE WALKTHROUGH — 8:00–8:45

[SCREEN]

Open IntelliJ / VS Code and show:

com.studenttracker

```
├── config
├── controller
├── dto
├── entity
├── exception
├── mapper
├── repository
├── security
├── service
│   └── impl
└── util
```

[SPEAK]

This is the backend package structure.

Each package has a specific responsibility.

The **controller** package contains REST controllers.

The **service** package contains business service contracts and implementations.

The **repository** package contains Spring Data repositories.

The **entity** package contains the JPA persistence models.

The **DTO** package contains request and response contracts.

The **mapper** package handles conversion between entities and DTOs.

The **security** package contains the JWT and authenticated-user infrastructure.

And finally, the **exception** package contains domain exceptions and global exception handling.

This separation makes the application easier to understand, test, maintain, and extend.

---

10. DATABASE & DOCKER — 8:45–9:15

[SCREEN]

Open Docker Desktop → MySQL container.

Then optionally show MySQL tables.

[SPEAK]

For persistence, I am using **MySQL running inside Docker**.

The database has persistent storage through a Docker volume, so the data isn't lost simply because the container is stopped.

The database schema is managed using **Flyway migrations**.

This means database changes can be version-controlled along with the application code.

For example, the authentication system currently uses tables such as:

app_users

roles

user_roles

The backend communicates with MySQL through **Spring Data JPA and Hibernate**.

---

11. FUTURE SCOPE — 9:15–9:45

[SCREEN]

Show dashboard / project structure.

[SPEAK]

The current implementation establishes the core foundation of the application.

The next phase will focus on the actual assignment-management workflow.

The planned features include:

- Assignment CRUD
- Course management
- Student management
- Assignment submission
- Submission history
- Teacher and administrator workflows
- Live dashboard data
- Deadline tracking
- Additional academic analytics

These features will be implemented incrementally on top of the existing authentication, database, and layered backend architecture.

---

12. CONCLUSION — 9:45–10:00

[SCREEN]

Return to dashboard or landing page.

[SPEAK]

So, to summarize:

Student Assignment Tracker is a full-stack application built using **React.js, Java Spring Boot, and MySQL**.

Through this project, I implemented a layered backend architecture, REST-based communication, JWT authentication, Spring Security, BCrypt password hashing, role-based authorization, JPA and Hibernate persistence, Flyway database migrations, and Dockerized MySQL.

The project is designed to be extended feature by feature as the assignment-management domain is developed further.

Thank you for watching my project demonstration.

QUICK DEMO CHECKLIST

Before starting the recording:

- Start Docker Desktop
- Run .\start-all.ps1
- Verify frontend at localhost:5173
- Verify backend at localhost:8081
- Open landing page
- Open login page
- Keep demo credentials ready
- Login successfully
- Open dashboard
- Open IDE
- Keep backend package structure ready
- Keep Docker/MySQL ready
- Close unnecessary browser tabs
- Increase IDE/terminal font size
- Start screen recording

DEMO CREDENTIALS

Email: student@example.com

Password: Student@123

Role: ROLE_STUDENT

IMPORTANT PRESENTATION RULE

Do not say that features are completed if they are currently only planned.

Use:

"The current implementation includes..."

and then:

"The next phase will implement..."

This makes the presentation technically accurate and professional.